Sprint 3 · Relación Idol–Cancion–Grupo y recorrido con apuntadores

Objetivo del sprint	Completar la relación entre `Idol`, `Cancion` y `Grupo` (popularidad con bono y líder) y recorrer el arreglo de integrantes con aritmética de apuntadores.
Etapas que cubre	Segunda parte de la Etapa 4 y primera parte de la Etapa 5
Puntos comprometidos	18 puntos en 6 historias
Incremento esperado	`manager.c` con la Etapa 4 completa y `mostrar_grupo`, `contar_activas` e `idol_mas_cansada` funcionando.

Sprint Backlog

ID	Historia	Tareas sugeridas	Pts	Responsable
HU-15	Popularidad total con bono de canción	Escribir `popularidad_total` · Probar sin canción · Probar con vigencia 0 en una canción compartida	3	3
HU-16	Definir el Grupo	Definir `Grupo` · Inicializar el grupo con tres integrantes · Verificar `n` y `lider`	2	2
HU-17	Cambiar la líder del grupo	Escribir `cambiar_lider` comparando direcciones · Probar los casos de rechazo · Imprimir `g.lider->fans`	5	5
HU-18	Mostrar el grupo recorriéndolo con apuntadores	Escribir `mostrar_grupo` con apuntador · Probar con grupo NULL · Probar el recorrido inverso	3	3
HU-19	Contar integrantes activas	Escribir `contar_activas` · Probar con una idol sin energía · Probar con grupo NULL	2	2
HU-20	Encontrar a la integrante más cansada	Escribir `idol_mas_cansada` · Probar con NULL y con grupo vacío · Llamar `descansar` con el resultado	3	3
		Total	**18**	18

Sprint Review: se acepta el sprint si…

- `popularidad_total` funciona con una idol sin canción y, si `discos[0].vigencia = 0;`, la canción deja de dar bono a todas las integrantes que la comparten.
- `cambiar_lider` devuelve 0 con NULL, con una idol fuera del grupo y con un lugar del arreglo que todavía no es integrante.
- `mostrar_grupo` recorre con apuntador, sin `[]`, y sigue funcionando al invertir el recorrido.
- `idol_mas_cansada(NULL)` devuelve NULL y `descansar` cambia a la integrante original dentro del grupo.

Riesgos y consejos

- Comparar nombres en lugar de direcciones en `cambiar_lider`.
- Olvidar la verificación `i->cancion != NULL` antes de `i->cancion->vigencia`.
- Usar `p <= g->miembros + g->n` (se pasa un elemento) en lugar de `p < ...`.

HU-15 · Popularidad total con bono de canción

Prioridad Alta · 3 pts

Como manager del grupo, **quiero** conocer la popularidad total de una integrante contando el bono de su canción solo si sigue vigente, **para** medir su verdadero impacto.

Criterios de aceptación

- `popularidad_total` devuelve `popularidad + bonus` solo si `cancion != NULL` y `cancion->vigencia > 0`.
- Una idol sin canción devuelve solo su popularidad, sin errores.
- Si se pone `discos[0].vigencia = 0;`, la canción deja de dar bono a **todas** las integrantes que la comparten.
- La condición verifica NULL antes del acceso encadenado `i->cancion->vigencia`.

Depende de: HU-14 · Sprint 3 · Entregable: `int popularidad_total(const Idol *i)`

HU-16 · Definir el Grupo

Prioridad Alta · 2 pts

Como manager del grupo, **quiero** agrupar hasta cuatro integrantes con un nombre y un contador, **para** administrar al grupo completo.

Criterios de aceptación

- `Grupo` tiene `nombre`, `Idol miembros[4]`, `int n` e `Idol *lider`.
- En `main` se crea un grupo (por ejemplo, “Lumina”) con tres integrantes, `n = 3` y `lider = NULL`.
- Las canciones se asignan a elementos del arreglo con `&g.miembros[i]`.
- `n` nunca es mayor que 4.

Depende de: HU-13 · Sprint 3 · Entregable: struct Grupo + grupo “Lumina”

HU-17 · Cambiar la líder del grupo

Prioridad Alta · 5 pts

Como manager del grupo, **quiero** designar a la líder del grupo solo entre sus integrantes reales, **para** evitar que la líder sea una idol ajena al grupo.

Criterios de aceptación

- `cambiar_lider` devuelve 1 y asigna `g->lider` cuando `nueva` es un elemento de `g->miembros` dentro de las primeras `n` posiciones.
- Devuelve 0 y no modifica nada si `g` es NULL, si `nueva` es NULL, si la idol está fuera del grupo (aunque tenga el mismo nombre) o si apunta a un lugar del arreglo con índice mayor o igual a `n`.
- La validación compara **direcciones**, no nombres.
- Tras `cambiar_lider(&g, &g.miembros[0])`, `g.lider->fans` imprime los fans de la primera integrante.

Depende de: HU-16 · Sprint 3 · Entregable: `int cambiar_lider(Grupo *g, Idol *nueva)`

Etapa 5 · Arreglos de structs y aritmética de apuntadores

Concepto: el nombre de un arreglo es un apuntador a su primer elemento, `miembros + i` apunta al elemento `i` y una función puede devolver un apuntador a un elemento. **Producto:** `mostrar_grupo`, `contar_activas`, `idol_mas_cansada` y `dar_concierto`. **Sprints 3 y 4.**

HU-18 · Mostrar el grupo recorriéndolo con apuntadores — Prioridad Media · 3 pts

Como manager del grupo, **quiero** ver a todo el grupo en pantalla, **para** revisar el estado de todas las integrantes de una vez.

Criterios de aceptación

- `mostrar_grupo` imprime el nombre del grupo y los datos de cada integrante usando `mostrar`.
- El recorrido usa un apuntador (`p++`, `p < g->miembros + g->n`) y **no** usa `[]`.
- Si `g` es NULL, la función regresa sin fallar.
- Si se cambia el orden del recorrido (de atrás hacia adelante), el programa sigue funcionando.

Depende de: HU-16 · Sprint 3 · Entregable: `void mostrar_grupo(const Grupo *g)`

HU-19 · Contar integrantes activas — Prioridad Media · 2 pts

Como manager del grupo, **quiero** saber cuántas integrantes siguen con energía, **para** decidir si el grupo puede presentarse.

Criterios de aceptación

- `contar_activas` devuelve cuántas integrantes tienen energía mayor que 0, usando `esta_activa`.
- Devuelve 0 si `g` es NULL o `n` es 0.
- En un grupo de 3 donde una tiene energía 0, devuelve 2.
- Recorre el arreglo con aritmética de apuntadores.

Depende de: HU-07, HU-16 · Sprint 3 · Entregable: `int contar_activas(const Grupo *g)`

HU-20 · Encontrar a la integrante más cansada — Prioridad Alta · 3 pts

Como manager del grupo, **quiero** identificar a la integrante con menos energía, **para** hacerla descansar antes de la presentación.

Criterios de aceptación

- `Idol *idol_mas_cansada(Grupo *g)` devuelve la **dirección** de la integrante con menor energía dentro de `g->miembros`.
- Devuelve NULL si `g` es NULL o si el grupo no tiene integrantes.
- Llamar a `descansar` con el resultado modifica a la integrante original dentro del grupo, no a una copia.
- `main` verifica que el resultado no sea NULL antes de usarlo.

Depende de: HU-06, HU-16 · Sprint 3 · Entregable: `Idol *idol_mas_cansada(Grupo *g)`